

Challenge - Episode 3

Terranova Software - Gaming Explorer

Sommario: A Web Application for exploring the world of video games through the RAWG REST APIs.

Indice

I Introduction **2**

II Project Objectives **4**

III Guidelines **6**

IV Functional Specifications **8**
IV.1 Mandatory features 8
IV.1.1 Home Page 8
IV.1.2 Basic search 8
IV.1.3 Advanced search 9
IV.1.4 Video game list 9
IV.1.5 Detail page 10
IV.1.6 Favorites 10
IV.1.7 Application state management 10
IV.1.8 Responsive Design 11
IV.2 Main RAWG APIs to use 11

V Technical Specifications **13**
V.1 Backend 13
V.2 Frontend 13
V.3 Database 14
V.4 Code Versioning 14
V.5 API Testing with Postman 14

VI Use of Artificial Intelligence in development **16**

VII Optional Part (Bonus) **18**
VII.1 User authentication 18
VII.2 Dark / Light Mode 19
VII.3 Video game comparison 19
VII.4 Wishlist 19
VII.5 Personal Statistics 19

VIII Presentation and Evaluation Criteria **20**
VIII.1 Evaluation criteria 21

IX References **22**

Capitolo I

Introduction

The Gaming Explorer project consists of building a Web Application dedicated to exploring the world of video games through integration with public REST APIs. The application will allow users to search for video games, view their main information, apply filters and sorting criteria, display detailed pages and manage a personal collection of favorite games.

Data will be retrieved using the RAWG Video Games Database API, which provides a large catalog of video games complete with images, descriptions, genres, platforms, ratings and other relevant information.

The project is designed as a complete development experience, in which students will build a modern application following a client-server architecture, integrating a backend developed in ASP.NET Core (or an equivalent framework) with a frontend built in Angular (or an equivalent framework). During development, students will apply the main concepts of software design, the use and integration of REST APIs, data management, code versioning and the creation of responsive user interfaces.

This document describes the mandatory features, the technical specifications and the way the project is to be carried out, and also proposes some Bonus implementations for students who wish to further enrich their application. In addition, it outlines the principles used to evaluate the projects.

The final goal is to build a complete, functional and well-structured Web Application, developed according to good software engineering practices.



Figure 1: Introduction to the Gaming Explorer app

Capitolo II

Project Objectives

The purpose of the Gaming Explorer project is to enable students to design and develop a complete Web Application, applying the knowledge acquired during the course in a practical context.

At the end of the activity, students should be able to analyze the requirements of an application, design a modular software architecture and build a solution that integrates a backend, a modern frontend and external services through REST APIs.

In particular, the project aims to develop the following skills:

- designing and building a Web Application following a modern client-server architecture;
- developing REST APIs using ASP.NET Core (or an equivalent framework);
- building a dynamic and responsive user interface using Angular, TypeScript and Bootstrap (or an equivalent framework);
- querying external REST APIs and processing responses in JSON format;
- implementing advanced search, filtering, sorting and data navigation features;
- managing data persistence through a relational database;
- applying good software development practices, favoring modularity, code reuse and separation of concerns;
- using Git for code versioning and for collaboration among group members;
- documenting the project and justifying the design and technology choices made;
- testing and validating APIs with dedicated tools, such as Postman;
- using Artificial Intelligence tools to support development, while maintaining a full understanding of the code produced and the solutions implemented.

The final goal is not merely to build a working application, but to develop a high-quality software project, characterized by readable and maintainable code, a polished user interface and a design consistent with modern development methodologies. Students will therefore be evaluated not only on the completeness of the features implemented, but also on the overall quality of the proposed solution, their ability to work as a team and their mastery of the technologies used.

Capitolo III

Guidelines

In the context of the Gaming Explorer project, these guidelines define the fundamental principles for developing a functional, modern and well-structured Web Application.

The functional aspects concern the features the application must offer to users, including searching for video games, viewing information, filtering results, displaying detail pages and managing favorite games. All features must be designed with particular attention to usability, ease of navigation and user experience.

The technical aspects, on the other hand, concern the technologies and tools used to build the project. The application must be developed according to a client-server architecture, with a backend handling requests and a frontend providing the user interface, integrating external REST APIs to retrieve data. Appropriate tools must also be adopted for code versioning, API testing and database management.

In short, the guidelines constitute a roadmap for carrying out the project, ensuring that students develop a complete, organized solution that complies with good software development practices.

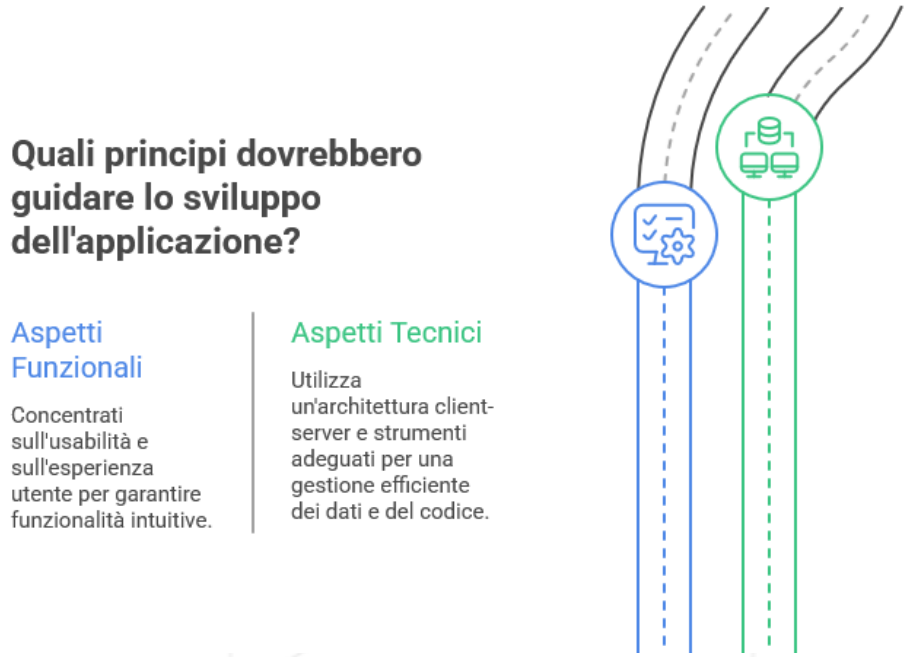


Figure 2: Guidelines for building the WebApp

Capitolo IV

Functional Specifications

The functional specifications describe the features that the Gaming Explorer Web Application must implement. The features are divided into mandatory and Bonus. The mandatory features are the minimum requirements for completing the project, while the Bonus features allow students to enrich the application with additional capabilities and obtain a higher grade.

The application must be designed with particular attention to the user experience, ensuring an intuitive interface, simple navigation and adequate response times. All features must be fully integrated and accessible through a modern, responsive graphical interface.

IV.1 Mandatory features

The Web Application must implement at least the following features.

IV.1.1 Home Page

The home page must present an overview of the application, including a search bar and a selection of video games retrieved through the APIs. The Home page must serve as the main entry point to the application's features.

IV.1.2 Basic search

The user must be able to search for a video game by entering all or part of its name. The search must query the REST APIs and display the results dynamically.

IV.1.3 Advanced search

The application must allow the search to be refined using one or more filters, including:

- video game genre;
- platform;
- release year;
- rating;
- sorting criterion (for example name, release date or rating).

Results must be updated either automatically or through a dedicated user action.

IV.1.4 Video game list

Search results must be presented as a list or a grid of Cards. For each video game, at least the following information must be displayed:

- cover image;
- title;
- release date;
- average rating;
- main platforms.

Each item in the list must be selectable.

IV.1.5 Detail page

Selecting a video game must open a dedicated page containing detailed information, such as:

- description;
- images and screenshots;
- genres;
- supported platforms;
- developer;
- publisher;
- rating;
- Metacritic score.



Metacritic is an American website that aggregates reviews of films, TV series, music, books and video games. Its main feature is the Metascore, a single score expressed out of 100, calculated as a weighted average of reviews written by the main publications and specialized critics worldwide. The Metacritic score is made available by the RAWG APIs.

IV.1.6 Favorites

The user must be able to add and remove video games from their favorites list. The favorites list must be viewable in a dedicated section of the application and may be stored in the database.

IV.1.7 Application state management

The application must correctly handle the main operating situations, providing at least:

- a data loading indicator (loading);
- handling of communication errors with the APIs;
- messages when a search returns no results.

IV.1.8 Responsive Design

The interface must adapt correctly to the main devices (desktop, tablet and smartphone), ensuring simple and consistent navigation regardless of screen resolution.

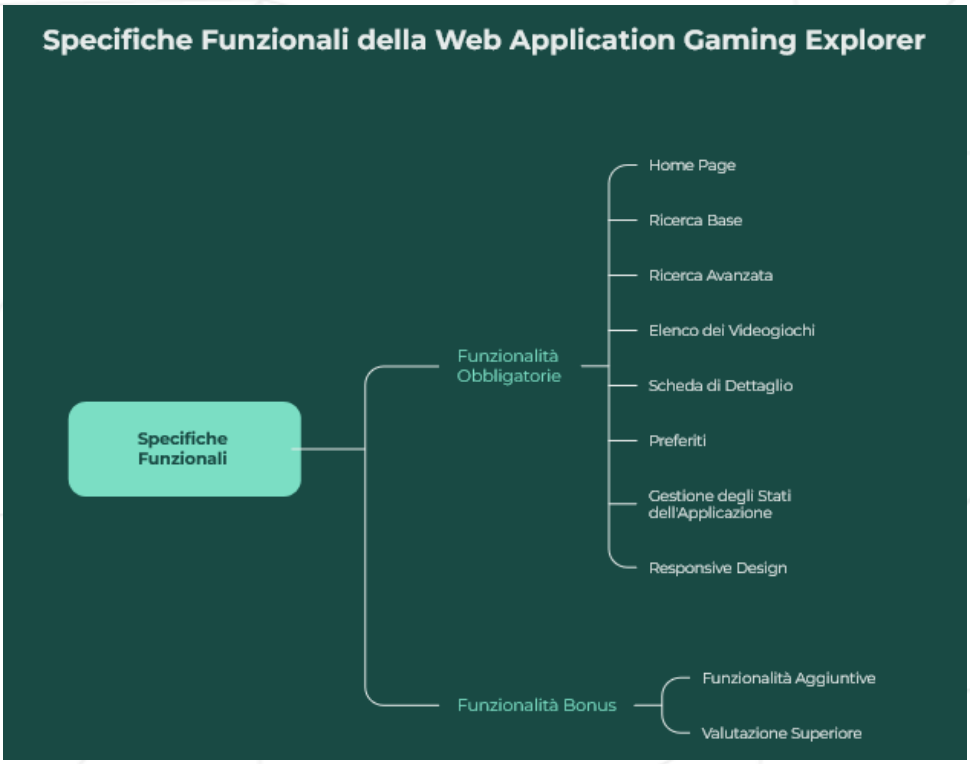


Figure 3: Functional Specifications

IV.2 Main RAWG APIs to use

To retrieve data, the application must use the RAWG Video Games Database APIs. The following table lists, non-exhaustively, the main endpoints to use for implementing the project's mandatory features.

Endpoint	Method	Description
`/games`	GET	Returns the list of video games. Supports search, filters, sorting and pagination.
`/games/{id}`	GET	Returns detailed information about a selected video game.
`/games/{id}/screenshots`	GET	Returns the screenshots associated with the video game.
`/genres`	GET	Returns the list of available genres.
`/platforms`	GET	Returns the list of supported platforms.

The main query parameters available for the `/games` endpoint are listed in the following table.

Parameter	**Description**
`search`	Searches for a video game by name.
`genres`	Filters results by genre.
`platforms`	Filters results by platform.
`dates`	Filters games by release date range.
`ordering`	Sorts results (name, date, rating, Metacritic, etc.).
`metacritic`	Filters games by Metacritic score.
`page`	Results page number.
`page_size`	Number of items returned per page.

The `/games` endpoint is the main access point to the application's data and makes it possible to implement most of the required features by appropriately combining search, filtering and sorting parameters. The endpoints dedicated to details, screenshots, genres and platforms complete the information needed to build the Web Application.

Capitolo V

Technical Specifications

The technical specifications define the technologies and tools to be used to build the Gaming Explorer project. The goal is to develop a modern, modular and easily maintainable Web Application, adopting tools widely used in professional development.

V.1 Backend

The application backend must be developed using C# and ASP.NET Core Web API, or equivalent frameworks with a justification for the choice.

The backend will be responsible for handling requests from the frontend, communicating with external APIs, processing the data received and providing REST services to the client through dedicated endpoints.

To develop the backend, students will use Visual Studio Community, a free development environment offering advanced tools for writing, debugging and testing .NET applications. Alternatively, they may use other development environments suitable for the implementation, justifying their choice.

V.2 Frontend

The frontend may be developed with Angular, using the TypeScript language, to build a modern, dynamic and responsive user interface.

For the graphic design, the use of the Bootstrap framework is recommended, in order to make it easier to build layouts that adapt to different devices.

Frontend development can be done with Visual Studio Code, using Node.js and npm (Node Package Manager) to manage the dependencies and tools needed to build the application.

Other frameworks and programming languages than those indicated in this section may nonetheless be used, with a justification for the choice.

V.3 Database

For persistent data management, the use of SQL Server Express, the free version of Microsoft's relational database, is suggested. Alternatively, other relational database management systems (RDBMS) may be used, such as:

- PostgreSQL;
- MySQL;
- SQLite.

The choice of a different database must be properly justified in the project's technical documentation.

If some information is stored exclusively on the client side (for example the favorites list), the use of the browser's Local Storage is permitted.

V.4 Code Versioning

Using Git as the version control system throughout the entire development of the project is mandatory.

Each group must keep the source code in a shared repository, preferably on GitHub, documenting the evolution of the project through frequent and descriptive commits.

Using Git will make it possible to:

- track changes made to the code;
- collaborate effectively among group members;
- manage different versions of the application;
- facilitate any review and debugging activities.

V.5 API Testing with Postman

Before integration with the frontend, all APIs developed in the backend must be verified using Postman (or equivalent software). Students must use this tool to:

- verify that the REST endpoints work correctly;
- check the HTTP status codes returned;
- analyze the responses in JSON format;
- validate input parameters;
- simulate error conditions and verify how they are handled.

It is also recommended to organize the requests in a Postman Collection, which may be submitted together with the project as technical documentation of the APIs developed.



Figure 4: Technical Specifications

Capitolo VI

Use of Artificial Intelligence in development

The use of Artificial Intelligence (AI) tools is permitted and encouraged throughout all phases of the project's development, as long as it serves as a support to learning and not as a substitute for the required skills.

Students may use tools such as ChatGPT, GitHub Copilot, Gemini, Claude or equivalent software for activities such as:

- requirements analysis;
- designing the application architecture;
- generating code snippets;
- explaining algorithms and libraries;
- finding and fixing errors (debugging);
- producing technical documentation;
- improving the quality of code and comments.

The use of Artificial Intelligence must always be accompanied by careful verification of the results produced. Generated code must be understood, validated and, if necessary, modified by the students before being integrated into the project.

During the final presentation and any discussion of the project, each group member must be able to:

- describe the application architecture;
- explain how the implemented features work;

- walk through the code developed;
- justify the design and technology choices made;
- make changes to the code if requested by the instructor.



The use of AI tools does not in itself constitute an advantage in the evaluation: the evaluation will be based on the quality of the project, the correctness of the solutions adopted and the students' ability to demonstrate a full understanding of the work done.

Finally, students are advised to use Artificial Intelligence responsibly, always checking the reliability of the information provided and consulting the official documentation of the technologies and APIs used when necessary.

Capitolo VII

Optional Part (Bonus)

The features described in this section are to be considered Bonus and are not mandatory requirements for completing the project. Students may freely choose whether to implement one or more of them, in order to enrich their Web Application and demonstrate skills beyond the minimum objectives.

Implemented Bonus features will be taken into account in the final evaluation, provided they are correctly integrated with the rest of the application, adequately documented and consistent with the project architecture.

Some possible extensions are listed below.

VII.1 User authentication

Implementation of an authentication system that allows users to register, log in and access a personal area. Authentication may be implemented using ASP.NET Core Identity, possibly integrated with JWT (JSON Web Token) for handling authentication in the REST APIs, or using other equivalent technologies, properly documented.

VII.2 Dark / Light Mode

Implementation of a system that lets the user choose between a light and a dark theme, preserving the selected preference across subsequent visits to the application.

VII.3 Video game comparison

The ability to select two or more video games and view their main characteristics on a single comparison screen, making it easier to compare platforms, ratings, release dates and other available information.

VII.4 Wishlist

Implementation of a list of video games the user wants to buy or try in the future, separate from the favorites list and manageable by adding and removing items.

VII.5 Personal Statistics

Implementation of a dashboard containing simple personal statistics derived from the use of the application, for example:

- number of video games viewed;
- number of favorite games;
- distribution by genre;
- distribution by platform;
- highest-rated video games.

The statistics may be calculated using the data stored in the database or in Local Storage.

Students are free to propose additional Bonus features not included in this list. Any proposals must be agreed upon with the instructor and be consistent with the project objectives and the technologies used.

Capitolo VIII

Presentation and Evaluation Criteria

The final project submission must include:

1. The complete source code, published in a Git repository (GitHub) with a meaningful commit history.

2. The technical documentation, containing a description of the implemented features, the application architecture, the technologies used and the main design choices.

3. A presentation (about 10 minutes) during which the group will illustrate the main features of the Web Application, the difficulties encountered during development and the solutions adopted.

During the presentation, each group member may be asked to describe parts of the project, explain how the developed code works and answer questions about the technologies used.

VIII.1 Evaluation criteria

The overall evaluation of the project will take into account both the technical quality of the application and the group's ability to illustrate its features during the presentation.

Criterion	**Score**
Correctness and completeness of the mandatory features	10 points
Code quality (organization, modularity, readability and good practices)	10 points
User interface, usability and responsive design	10 points
Technical documentation and Git repository	5 points
Project presentation and ability to explain design choices	5 points
Bonus features	+2 points

Bonus features do not replace the mandatory requirements: they will be considered only after verifying the correct implementation of the minimum required features.

Particular attention will also be paid to the overall quality of the project. A stable, well-organized, easy-to-use application with clear and documented code will be evaluated positively.

The use of Artificial Intelligence tools is not an evaluation criterion.

During the presentation, students must demonstrate a full understanding of the code they produced, regardless of the tools used during development.

Capitolo IX

References

Tool	URL
Visual Studio Community; Visual Studio Code; GitHub	https://visualstudio.microsoft.com/free-developer-offers/
SQL Server Express	https://www.microsoft.com/en-us/sql-server/sql-server-downloads
Postman	https://www.postman.com/
npm.js	https://www.npmjs.com/
GDPR	https://www.garanteprivacy.it/regolamentoue
Video Game API	https://rawg.io/apidocs
Visual Studio Code and TypeScript guide	https://kinsta.com/blog/typescript-visual-studio/